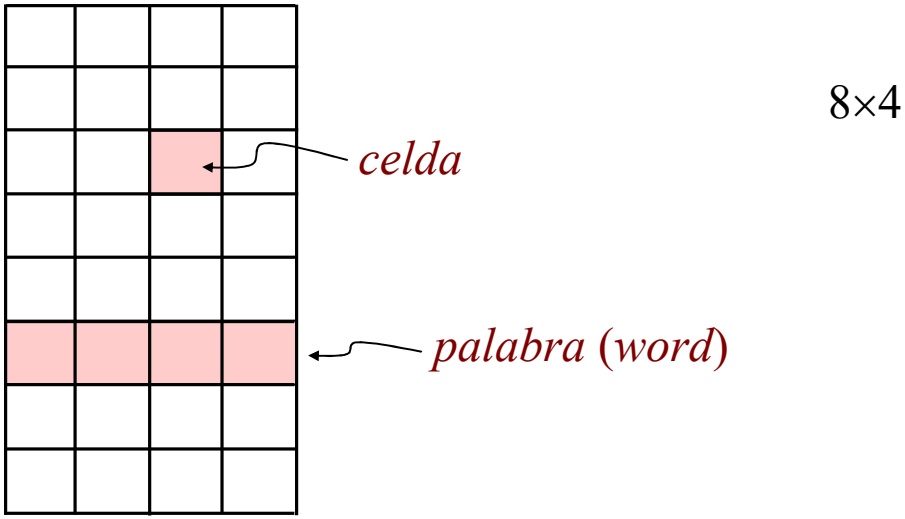
Memorias semiconductoras

Conceptos básicos:

- Una *memoria* guarda valores lógicos (*bits*) o números binarios, si se considera la analogía $1_2 \equiv 1_{lógico}$, $0_2 \equiv 0_{lógico}$.

- A *nivel funcional* podemos considerar que una memoria está formada por una *matriz de celdas*, de modo que en cada *celda* se guarda 1 bit (un dígito binario)



8×4

El tablero es el Mundo, las piezas son los fenómenos del Universo, las reglas del juego corresponden a lo que llamamos leyes de la Naturaleza. El jugador del otro lado está oculto, sabemos que juega limpio, genuina y pacientemente. Sin embargo, también sabemos que, a nuestra costa, nunca perdona un error u otorga la más mínima concesión a la ignorancia.

Thomas Henry Huxley

- Las memorias están diseñadas para manejar *información* (datos) en grupos de bits denominados *palabras* (*words*).

- En cada fila de la matriz de celdas se guarda 1 *palabra* (*word*).

- Según la memoria que se considere, el tamaño de las *palabras* (≡ número de columnas de la matriz de celdas ≡ número de bits que forman una *palabra*) puede ser de 1 bit, 4 bits, 8 bits, 9 bits o un múltiplo entero de 8.

- *Capacidad de una memoria*: indica el número total de bits que puede guardar dicha memoria. Los fabricantes acostumbran a indicar la capacidad de las memorias como el producto del *número de palabras* que pueden guardar por el *número de bits* que tiene una *palabra*. Así, por ejemplo, una memoria ROM 512×4 guarda 512 *palabras* de 4 bits (512 *nibbles*) o, si se prefiere, 2048 bits (≡ la matriz de celdas tiene 512 filas y 4 columnas… tiene 512×4 = 2048 celdas)

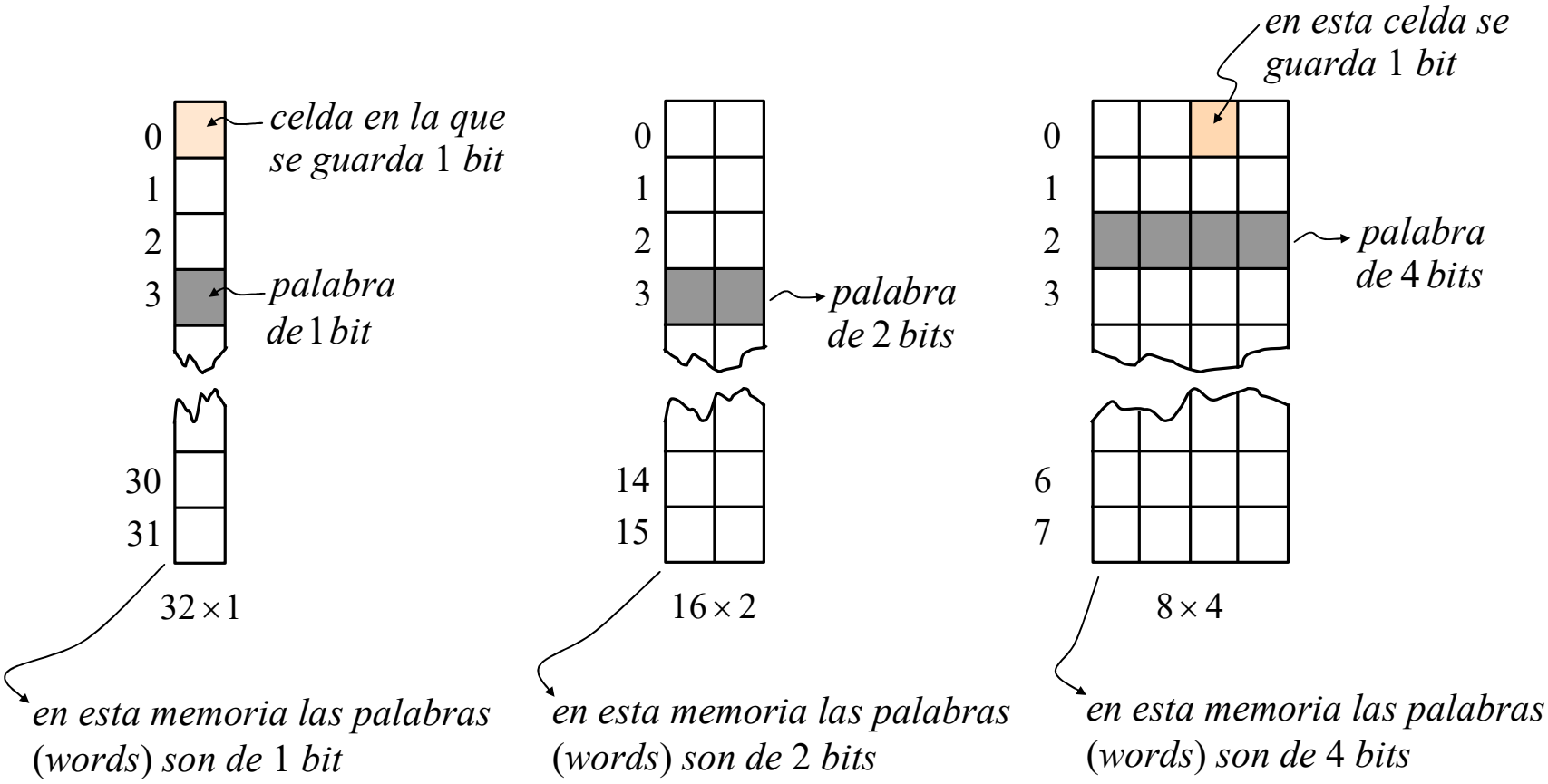
Nota:

· un grupo de 4 bits de denomina *nibble*

· un grupo de 8 bits se denomina *byte* y se indica con una *B* mayúscula.

· un bit se representa con una letra *b* minúscula.

Ejemplos de organizaciones de una memoria con una capacidad de 32 bits



Por cuestiones de optimización de los circuitos, el número de palabras que puede guardar una memoria *siempre* va a ser una potencia entera de la base 2. Esto ha hecho que se hayan adoptado las siguientes unidades (sufijos) para indicar la capacidad de las memorias:

Electrónica Digital	*Física*
$1k_{(kilo)} \equiv 2^{10} = 1024_{10}$	$1k \equiv 10^3$
$1M_{(mega)} \equiv 2^{20} = 1048576_{10}$	$1M \equiv 10^6$
$1G_{(giga)} \equiv 2^{30} = 1073741824_{10}$	$1G \equiv 10^9$
$1T_{(tera)} \equiv 2^{40} = 2^{20} \times 2^{20}$	$1T \equiv 10^{12}$

potencia entera de 2 más próxima a 10^{12}

Para evitar la confusión con los sufijos *k* (kilo, 10^3), *M* (mega, 10^6), *G* (giga, 10^9), etc. que se utilizan en otros campos de la Ingeniería, en Física, en Química, etc., en enero de 1999 el IEC (*International Electrotechnical Commission*) publicó la norma IEC 60027-2 en la que se establecen nuevos sufijos para representar potencias enteras de 2. Dichos sufijos fueron adoptados en 2002 por el IEEE en la norma IEEE Standard 1541-2002. En éstas normas se recomienda utilizar los siguientes sufijos:

$$1ki_{(kibi,kilobinary)} \equiv 2^{10} = 1024_{10}$$

$$1Mi_{(mebi,megabinary)} \equiv 2^{20} = 1048576_{10}$$

$$1Gi_{(gibi,gigabinary)} \equiv 2^{30} = 1073741824_{10}$$

$$1Ti_{(tebi,terabinary)} \equiv 2^{40} = 2^{20} \times 2^{20}$$

no

Ejemplo:

$$4ki = 4 \cdot 1ki = 2^2 \cdot 1 \cdot 2^{10} = 2^{12} = 4096_{10}$$

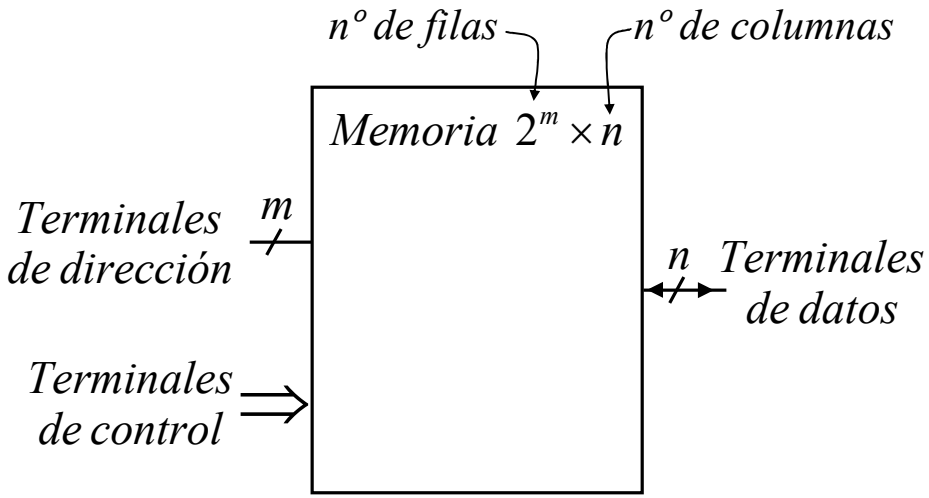
Nota: hay que tener presente que la terminología *Ki*, *Mi*, *Gi*, etc. no está muy difundida. En la mayoría de los documentos técnicos aún se utilizan los sufijos indicados en la diapositiva 4…. y no parece que la cosa vaya a cambiar pronto !!!.

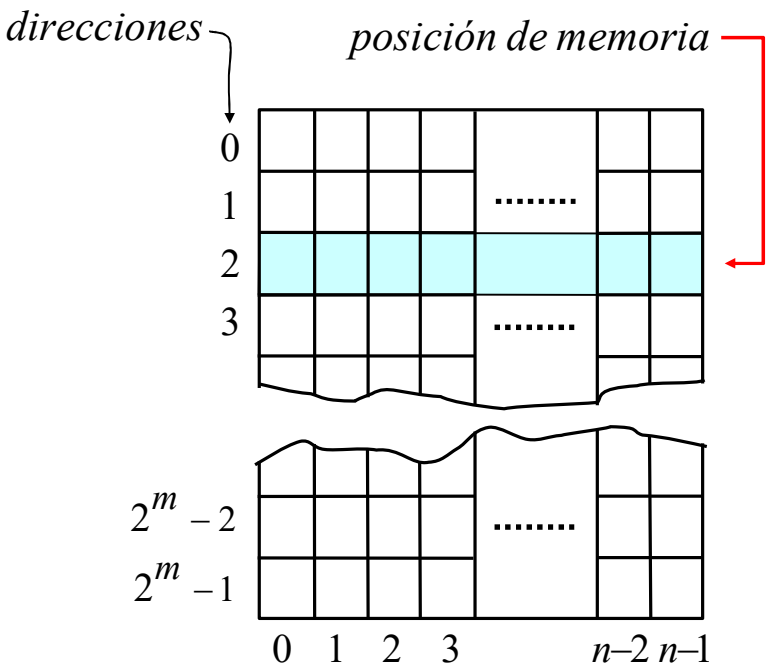
Pregunta: *¿Cómo sabemos cuando k representa* 10^3 *y cuando representa* 2^{10}? si

Pregunta: *¿Cuál es la diferencia entre* $1kb$ *y* $1kB$?

Pasatiempo: *¿Cuál es la diferencia entre* $1K$ *y* $1k$?

▪ Terminales básicos de una memoria de *pequeña capacidad*





_ La memoria indicada tiene 2^m posiciones (*filas*). Cada posición (*fila*) tiene asociada una *dirección* (≡ un número binario). La posición (*fila*) en la que se *lee* o se escribe (*guarda*) un dato (*word* o *palabra* de *n* bits) se determina decodificando la dirección presente en los *terminales de dirección*.

_ En cada posición (dirección) de la memoria se guardan *n* bits (un dato).

_ Tanto en el interior de un procesador como en la placa base de un PC, los terminales de dirección de una memoria se conectan al *bus de direcciones* y los terminales de datos se conectan al *bus de datos*.

▪ **Operación de lectura**: una operación de lectura consiste en leer los bits guardados en una de las posiciones de la memoria (≡ se lee el dato guardado en una fila de la matriz de celdas). Para realizar una *operación de lectura* hay que hacer lo siguiente:

1º: En los *terminales de dirección* hay que poner la dirección (número) correspondiente a la posición de memoria en la que está guardado el dato que se quiere conocer (leer).

2º: A continuación hay que configurar los *terminales de control* para realizar una *operación de lectura* y, un tiempo después, en los *terminales de datos* aparecen los bits guardados en la posición de memoria indicada en los *terminales de dirección*

Nota: el número y el tipo de *terminales de control* depende de la memoria que se considere. En general, los terminales de control permiten establecer la operación a realizar (*lectura* o *escritura*), conectar eléctricamente los terminales de datos al *bus de datos*, inhibir las operaciones de lectura y escritura, proteger los datos guardados por hardware, poner la memoria en *standby* (≡ modo de bajo consumo), etc.

Nota: cuando se habla de la velocidad/rapidez de una memoria se hace referencia al tiempo que tarda en realizar una operación de lectura y/o de escritura.

▪ **Operación de escritura**: en una memoria de capacidad $2^m \times n$, una operación de escritura consiste en guardar n bits en una de las 2^m posiciones (*filas*) de la memoria (≡ se guarda 1 bit en cada una de las celdas de una determinada fila de la matriz de celdas).

Para realizar una operación de escritura hay que hacer lo siguiente:

1º: En los *terminales de dirección* hay que poner la dirección correspondiente a la posición de memoria (≡ fila de la matriz de celdas) en la que se quiere guardar (escribir) un dato de n bits.

2º: En los *terminales de datos* hay que poner el dato de n bits que se quiere guardar en la posición de memoria (*fila*) indicada por el valor (dirección) presente en los terminales de dirección.

3º: Hay que configurar los *terminales de control* para que se realice una operación de *escritura* y, un tiempo después, en la posición de memoria indicada queda guardado el dato presente en los terminales de datos de la memoria.

Clasificación de las memorias semiconductoras

- *Memorias*
  - *Acceso Directo (RAM)*
    - *Volátiles o no pasivas (RAM)*
      - *SRAM*
        - *ASRAM ó SRAM*
        - *SSRAM* •
      - *DRAM*
        - *DDR3 SDRAM*
        - *DDR4 SDRAM*
        - *DDR5 SDRAM* •
        - *RDRAM*
    - *No volátiles o pasivas (ROM)*
      - *No borrables* •
        - *Mask ROM ó ROM*
        - *PROM*
          - *Fusibles*
          - *OTP ROM*
      - *Borrables*
        - *EEPROM*
        - *FLASH* •
        - *NVRAM*
        - *FRAM*
        - *PCRAM*
        - *etc.*
  - *Acceso Secuencial*
    - *FIFO ó tubo*
    - *LIFO ó pila*

Nota:

RAM (*Random Access Memories*)

ROM (*Read Only Memories*)

Definición RAE de aleatorio, ria

Del lat. aleatorius, der. de alea 'juego de azar', 'azar, suerte'.

1. adj. Que depende del azar (‖ casualidad). Un proceso aleatorio.

2. adj. Perteneciente o relativo al juego de azar.

En las memorias RAM (*Random Access Memories*) no hay nada aleatorio o que tenga que ver con el azar.

SRAM (*Static RAM*). Características básicas

- Se utilizan *biestables* para implementar las celdas (1 biestable/celda).

- El circuito asociado a cada celda tiene un tamaño (volumen) elevado ⇒ se pueden integrar 'pocas' celdas (biestables) en un chip ⇒ sólo se pueden construir memorias (chips) de baja capacidad (que contengan pocos biestables) ($V_{biestable} = 4 \cdot V_{condensador}$)

- $t_{\text{operación lectura}} \approx t_{\text{operación escritura}}$

- Son las memorias más rápidas ≡ son las memorias en las que las operaciones de lectura y de escritura se realizan en menos tiempo.

- Se utilizan para implementar las memorias *caché* en los PCS.

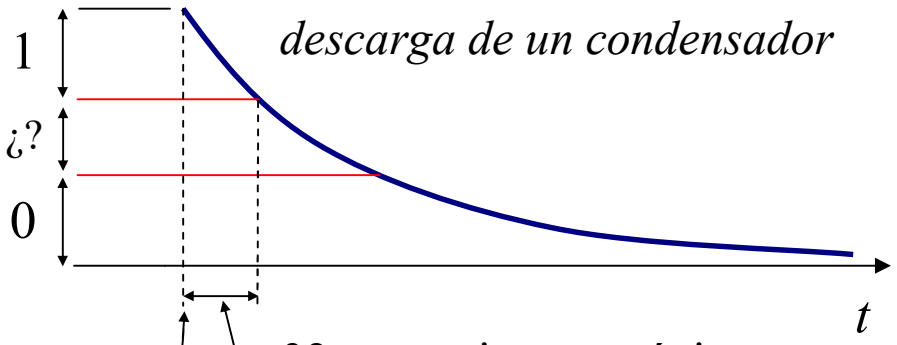
- Son volátiles (si se interrumpe el suministro de energía eléctrica pierden la información que guardan)

DRAM (*Dynamic RAM*). Características básicas

- Se utilizan *condensadores* para implementar las celdas (1 condensador/celda).

- El circuito asociado a cada celda tiene un tamaño (volumen) muy pequeño $\Rightarrow$ se pueden integrar muchos condensadores ($\equiv$ celdas) en un chip $\Rightarrow$ se pueden fabricar memorias (chips) con una capacidad elevada (contienen muchos condensadores)

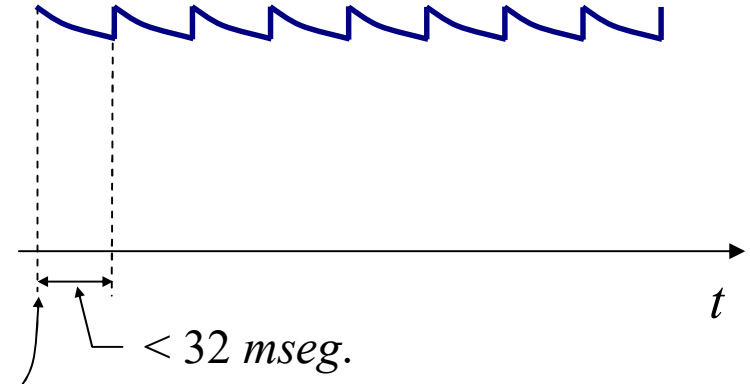
- Los condensadores se descargan con el paso del tiempo $\Rightarrow$ es necesario reestablecer (refrescar) su carga eléctrica cada pocos milisegundos ($\approx$32 *mseg.*) para que no se pierda la información guardada. Esta característica junto con el hecho de que los condensadores no se cargan instantáneamente hace que las memorias DRAM sean memorias bastante más lentas que las memorias SRAM. No hay que olvidar que durante las operaciones de refresco no se pueden realizar operaciones de lectura o de escritura.



32 *mseg*. tiempo máximo en recargar (refrescar) el condensador

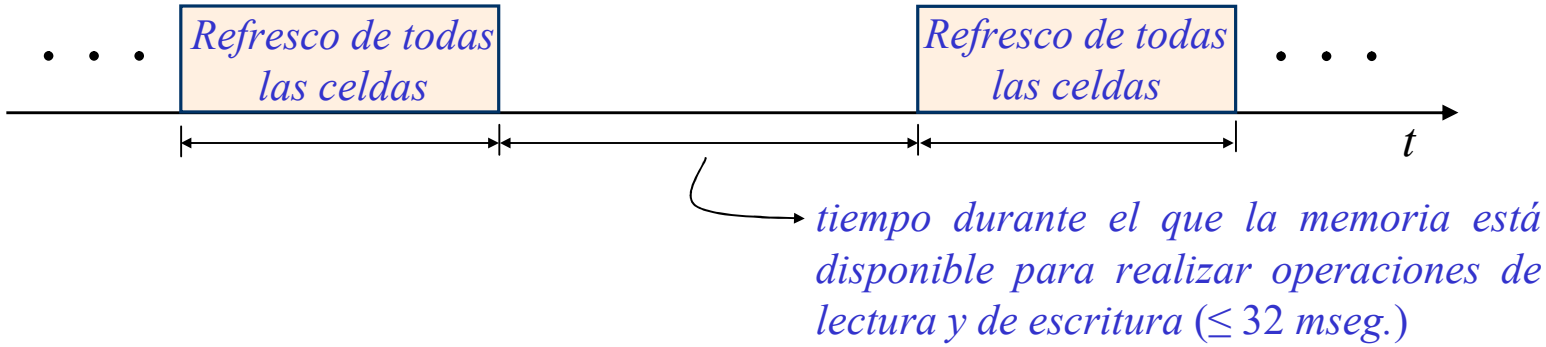
instante en el que se guarda un 1 en una celda
(se carga el condensador)

refresco de una celda (*recarga de un condensador*)

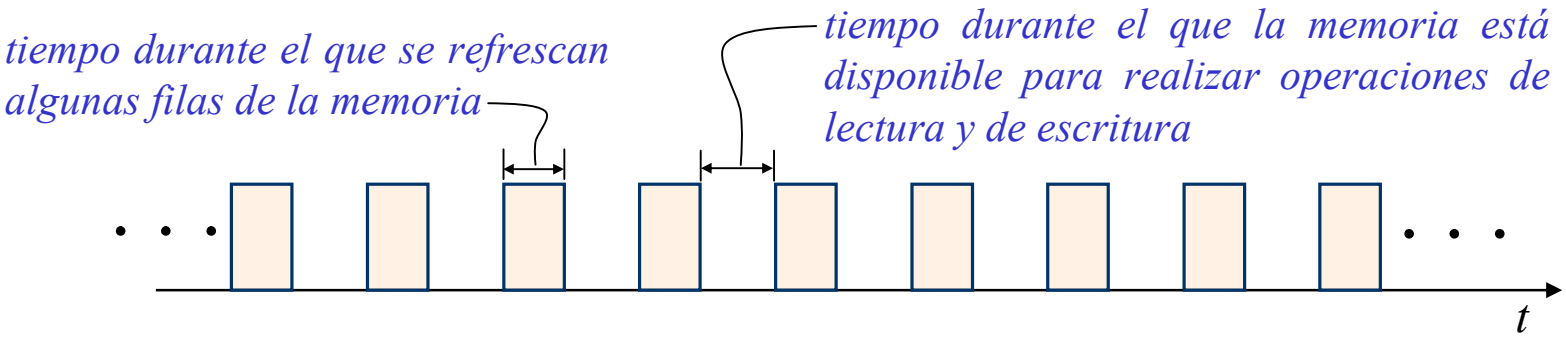


instante en el que se realiza una operación de refresco
(se recarga el condensador)

Modo de refresco: *ráfaga* (*Burst*)



Modo de refresco: *distribuido* (*Distributed*)



Nota: el JEDEC establece que el tiempo máximo que se puede tardar en refrescar una fila de una memoria DRAM es igual a 64 *mseg*. (32 *mseg.* para $T > 85$ºC). *JESD79-3F*, JEDEC Committee JC-42.3 Std. DDR3, 2010. El controlador de memoria se encarga de realizar las operaciones de refresco.

- $t_{\text{medio operación lectura}} \approx t_{\text{medio operación escritura}}$
- Se utilizan para implementar la *memoria principal* en los PCS (ordenadores).
- Son volátiles (si se interrumpe el suministro de energía eléctrica pierden la información que guardan)

FLASH. Características básicas

- Son memorias *no volátiles*. Es decir, no pierden la información que guardan si se interrumpe el suministro de energía eléctrica.

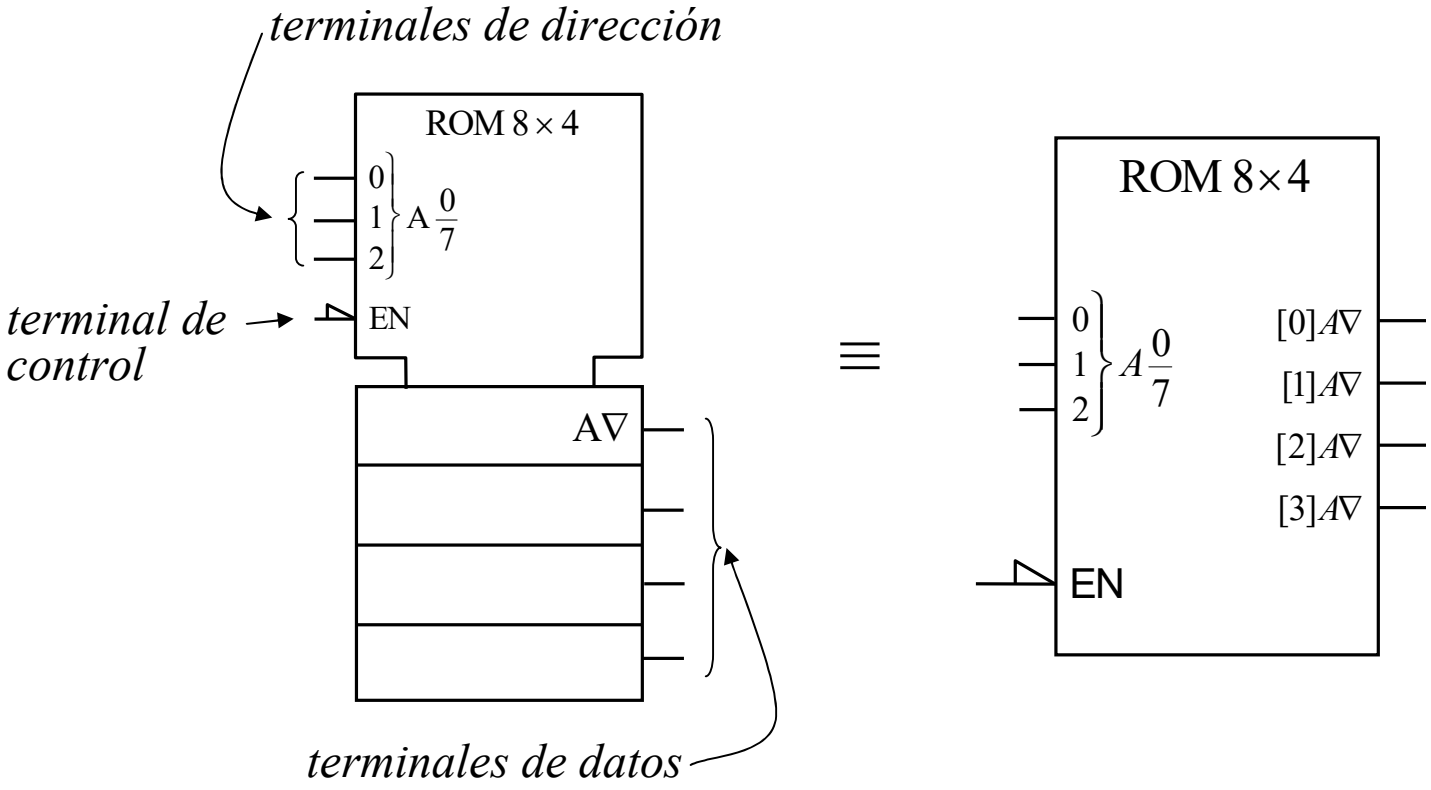
- $t_{\text{operación lectura}} <<< t_{\text{operación escritura}}$ (una operación de lectura dura mucho menos que una operación de escritura)

- Aunque a día de hoy se comercializan chips que contienen memorias Flash con una elevada capacidad, su elevado tiempo de escritura impide que se puedan utilizar para sustituir a las memorias SRAM o DRAM. A pesar de ello, las memorias Flash tienen un importante campo de aplicación comercial. Entre sus usos se encuentran los *pendrives*, los discos duros de estado sólido (*solid state drives*), la *BIOS* de los *PCS*, etc.

Nota: el término "*endurance*" se refiere al número de ciclos de *borrado/escritura* que puede soportar un bloque de una memoria Flash sin estropearse (typ: 10^6)

Nota: 'el *dorado*' es una memoria *no volátil*, con la *capacidad* de una memoria DRAM y con el *tiempo de acceso* de una memoria SRAM.

Ejemplo ROM



Nota: en simbología normalizada IE[3] Std. 91-1984:

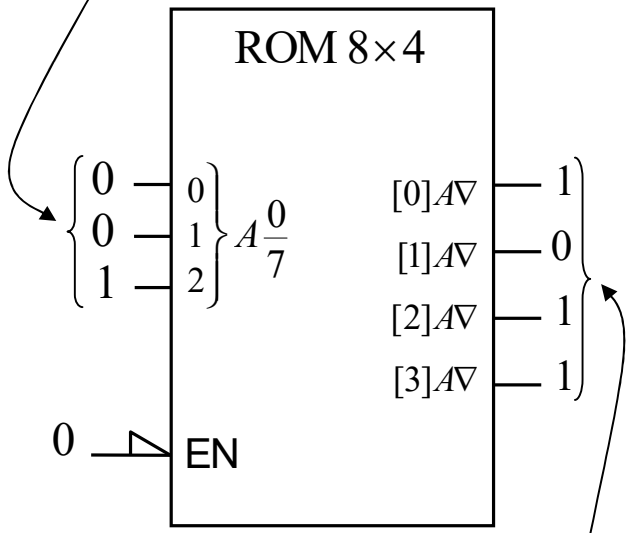
_ **A** indica dirección (*Address*)

_ **EN** (*enable*) indica permitir, habilitar, activar, etc.

_ ∇ indica terminal con tercer estado (≡ estado de alta impedancia)

Ejemplo ROM

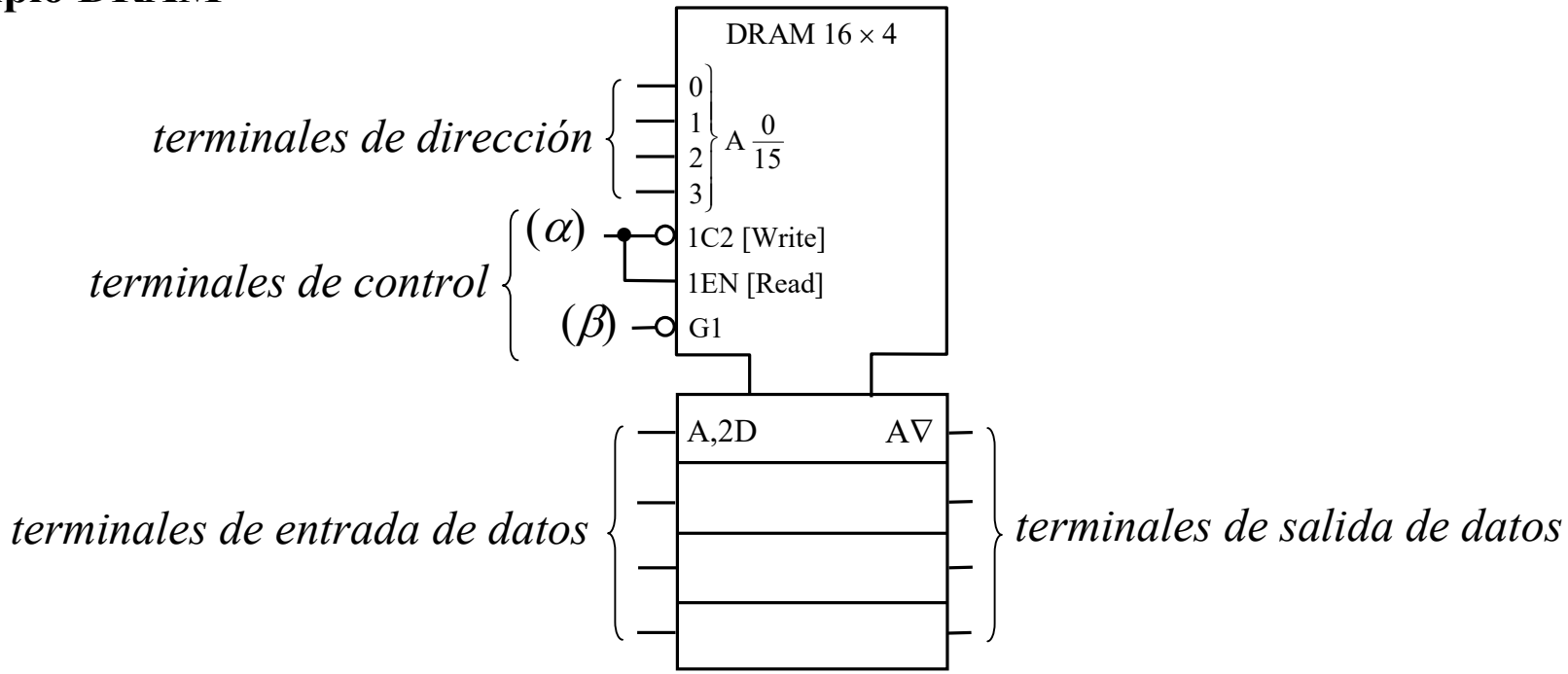
se indica (*se selecciona*) *la dirección* (*fila*) 100_2



direcciones			*contenido memoria*			
0	0	0	0	1	1	0
0	0	1	1	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	0
1	0	0	1	1	0	1
1	0	1	0	1	0	0
1	1	0	0	1	0	1
1	1	1	0	1	1	1

valor guardado en la posición de memoria 100_2

Ejemplo DRAM



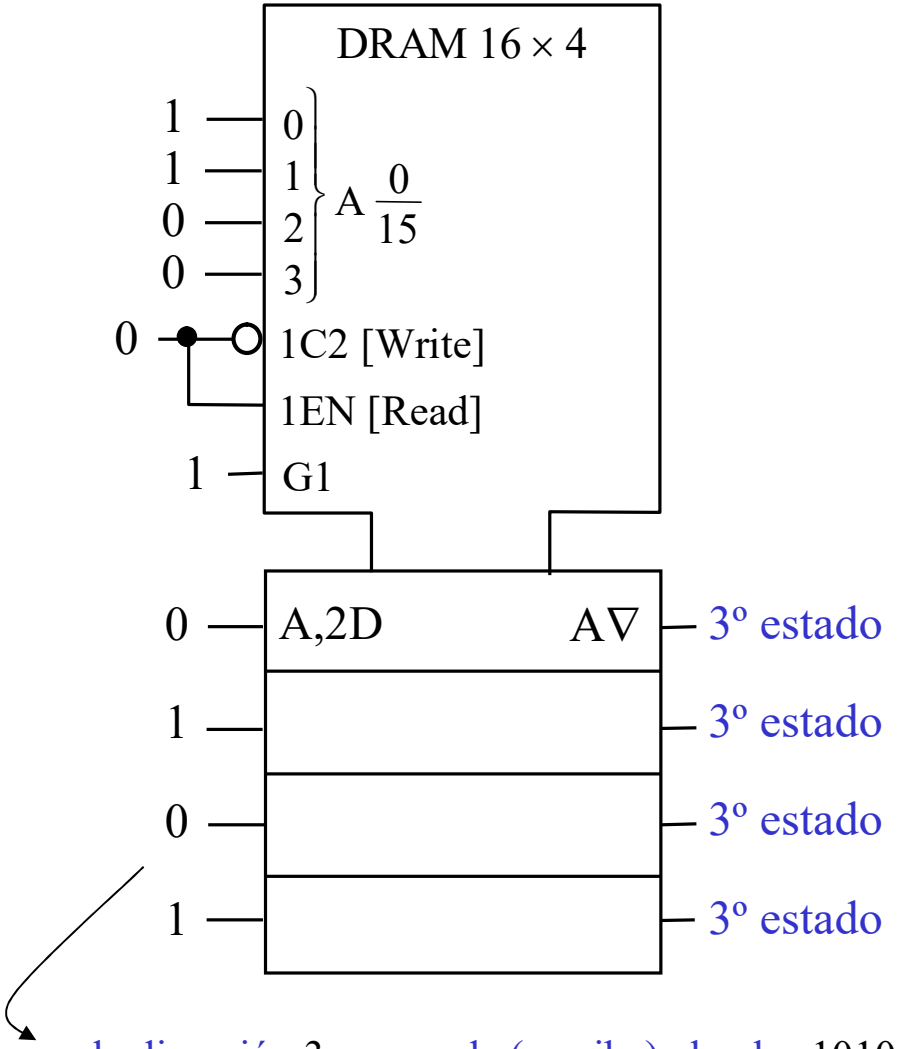
Notas:

$(\beta, \alpha) = 0\ 0 \rightarrow$ se realiza una operación de escritura [los terminales de salida de datos están en tercer estado (EN = 0)]

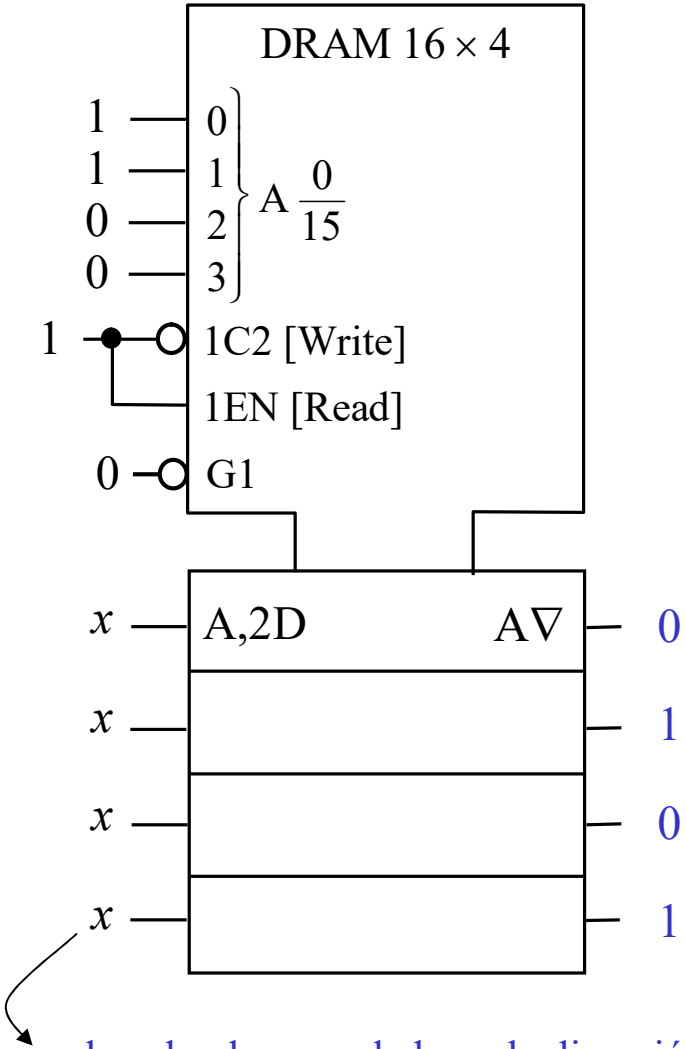
$(\beta, \alpha) = 0\ 1 \rightarrow$ se realiza una operación de lectura

$(\beta, \alpha) = 1\ x \rightarrow$ no se permite realizar operaciones de lectura ni de escritura (G1 = 0).

Ejemplos:

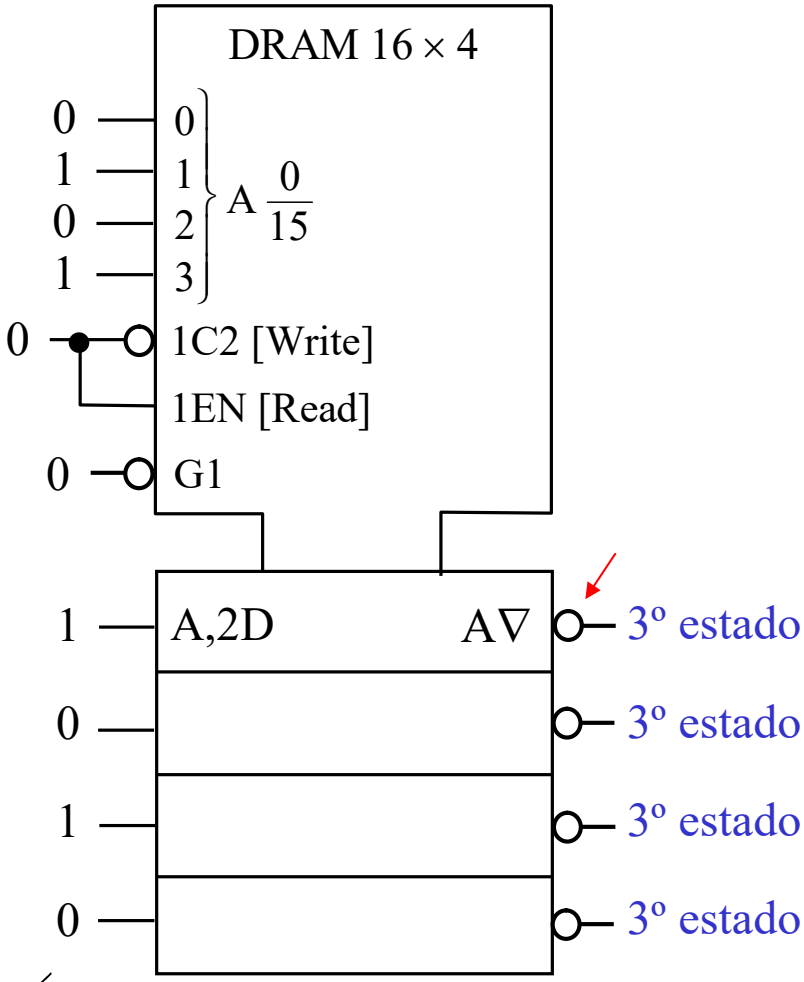


en la dirección 3 se guarda (escribe) el valor 1010

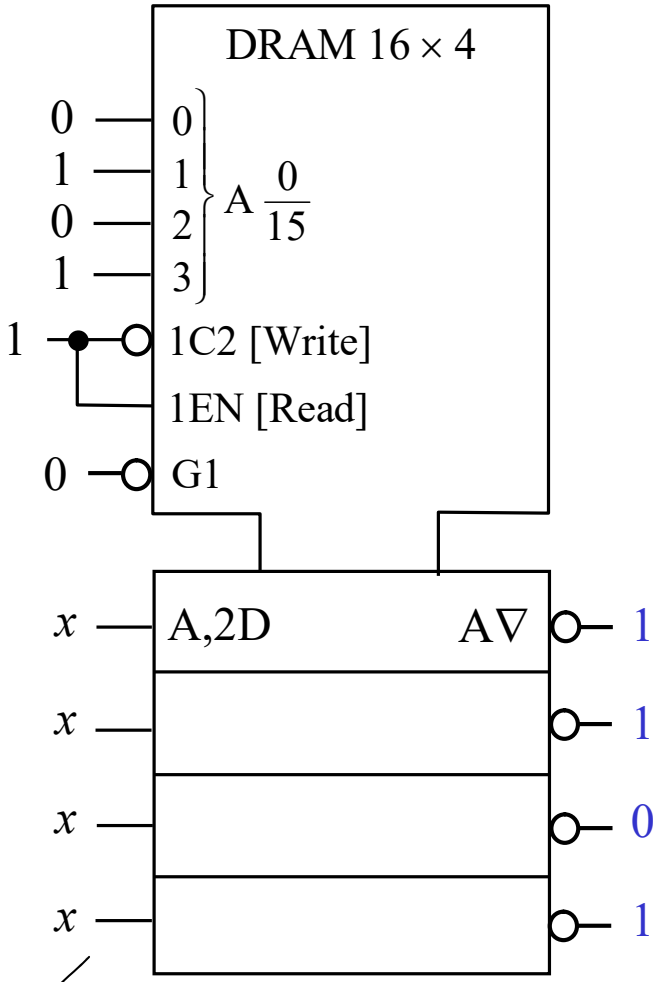


se lee el valor guardado en la dirección 3 y que resulta ser igual a 1010

Ejemplos:

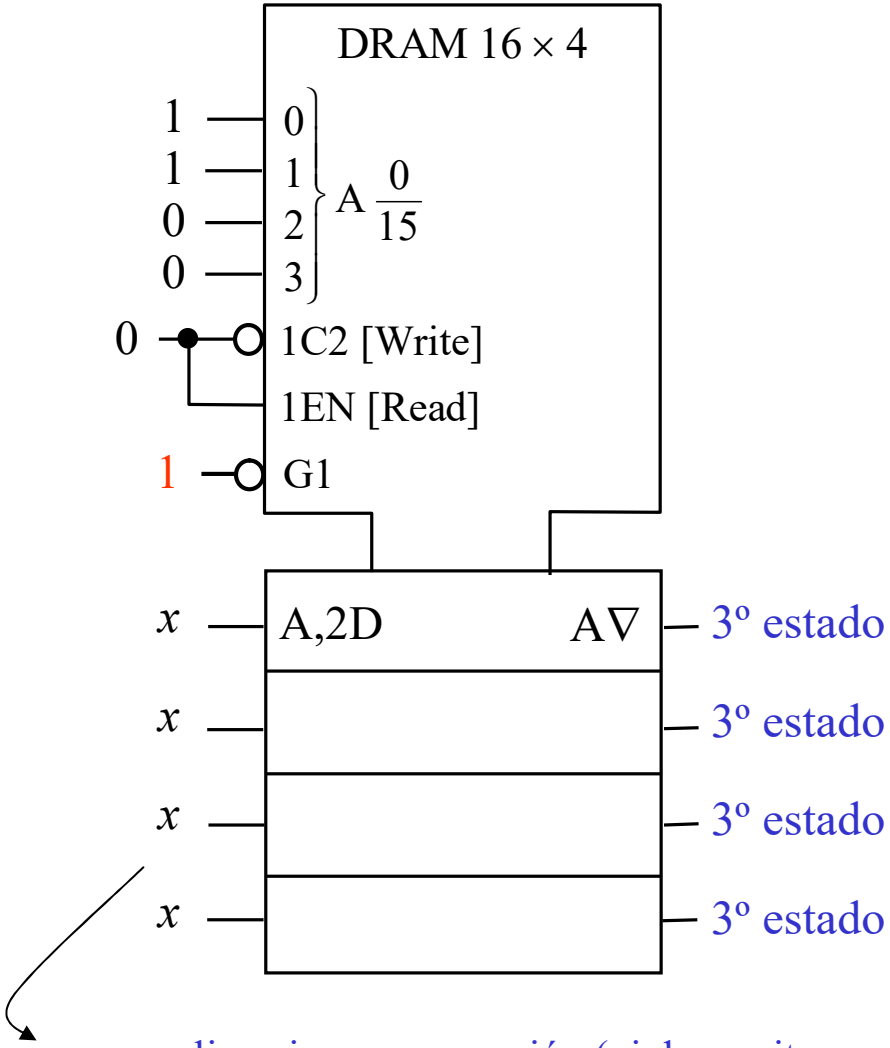


en la dirección 1010 se guarda (escribe) el valor 0101



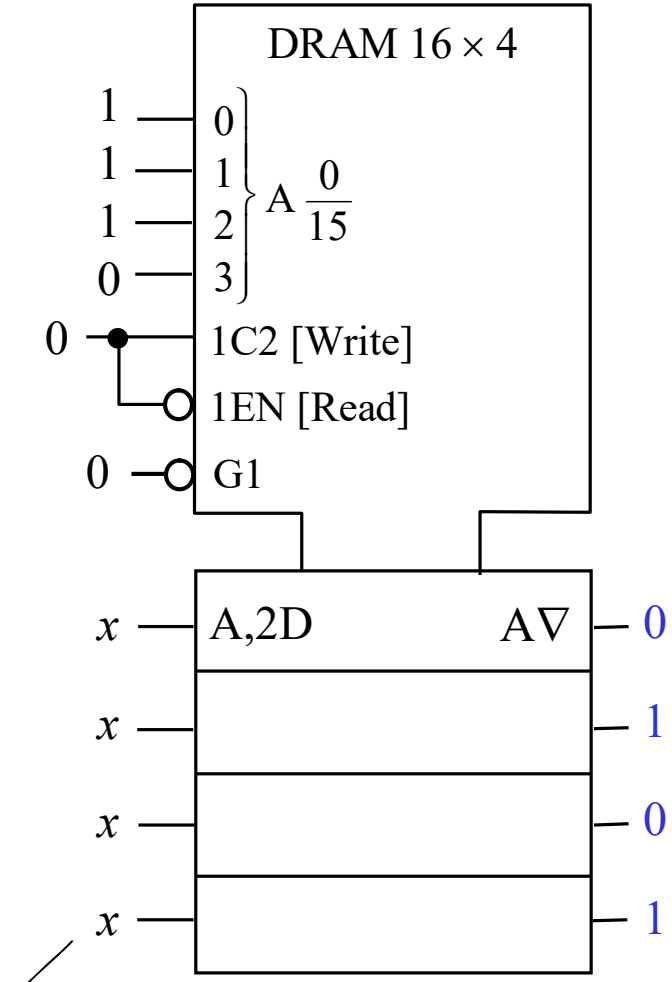
Se lee el valor guardado en la dirección 1010. En dicha dirección está guardado el valor 0100

Ejemplos:

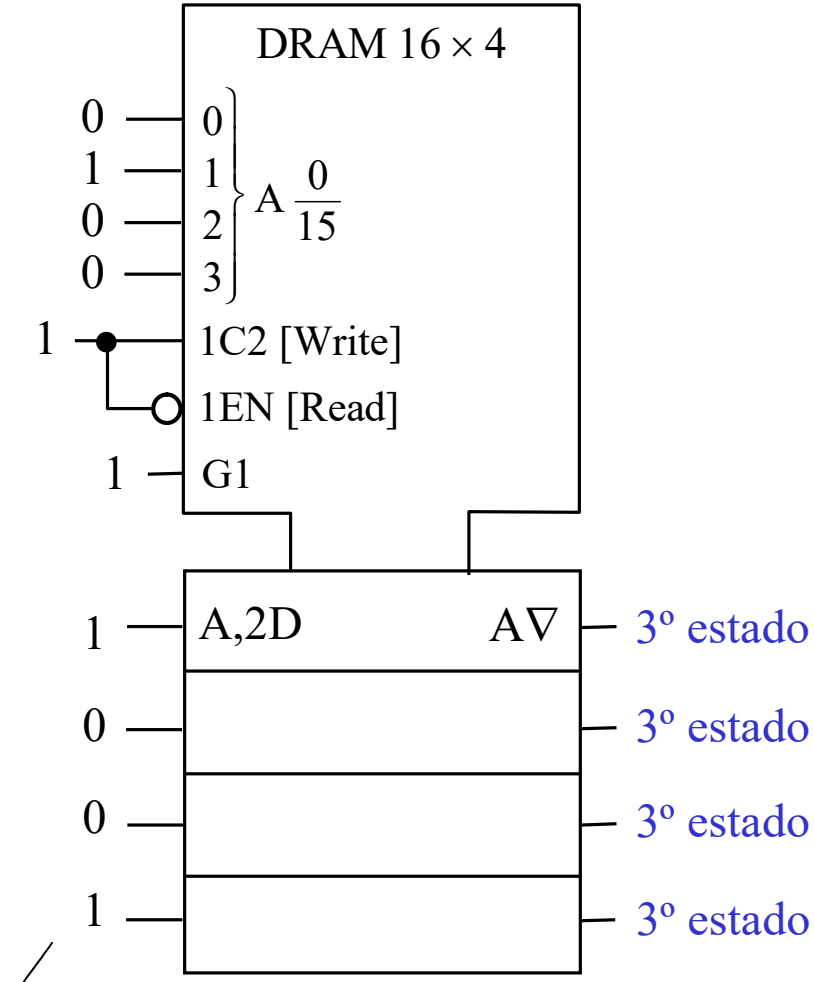


no se realiza ninguna operación (ni de escritura ni de lectura)

Ejemplos:



se realiza una operación de lectura en la dirección 0111, en la que hay guardado el valor 1010.



se realiza una operación de escritura: en la dirección 0010 se guarda el valor 1001

Memorias de doble puerto: en el mercado se pueden encontrar memorias RAM que permiten realizar simultáneamente,

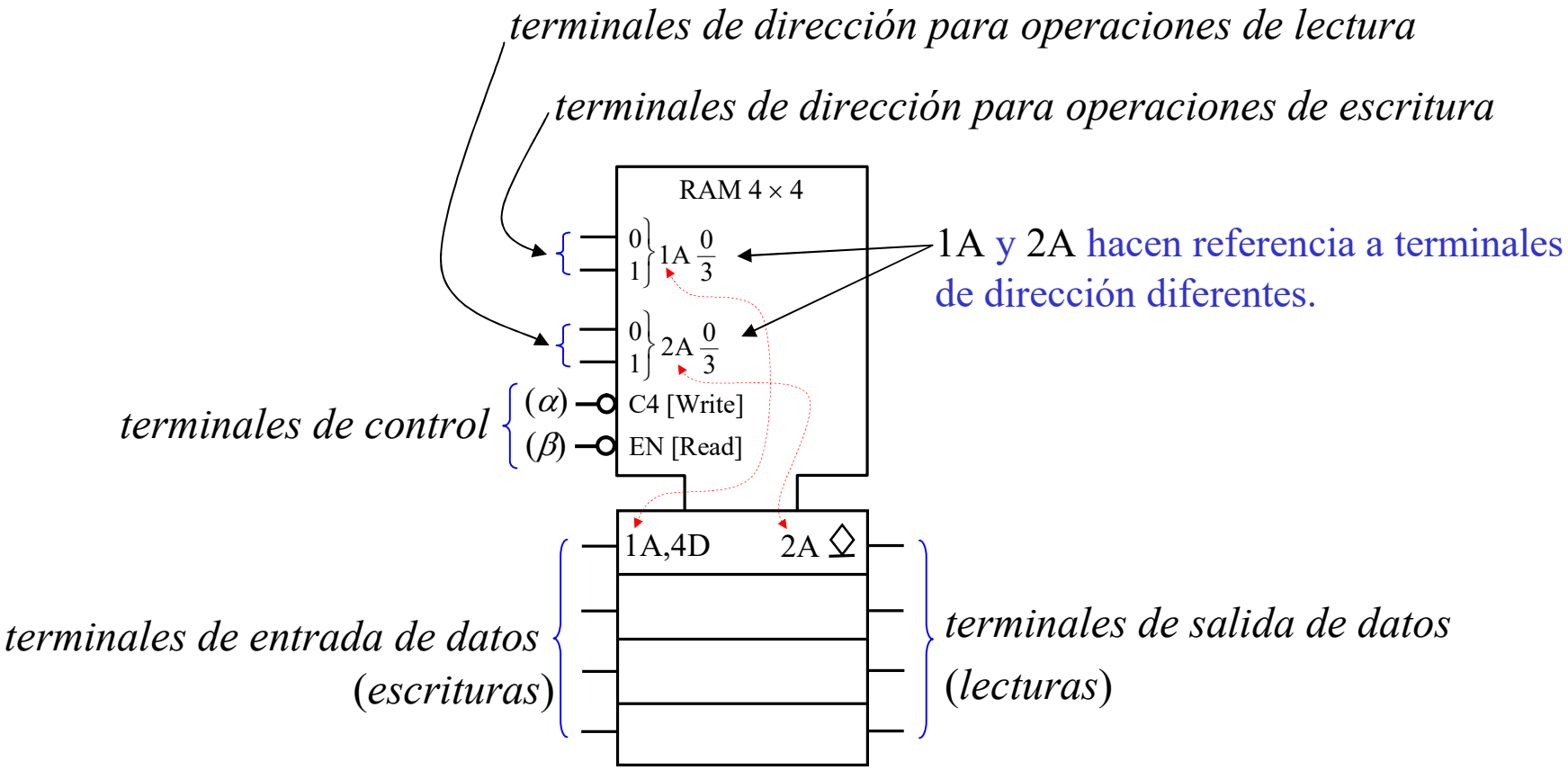
_ en una misma dirección: dos lecturas

_ en diferentes direcciones: dos lecturas, una lectura y una escritura o dos escrituras

En el mercado se pueden encontrar memorias de doble puerto tanto asíncronas como síncronas (Renesas, Cypress, etc.). La ventaja de este tipo de memorias reside en que presentan un *ancho de banda* considerablemente mayor que las memorias de un solo puerto (del orden del doble).

Nota: se define el *ancho de banda* de una memoria como el número de operaciones (de lectura) que puede realizar por segundo. Se suele expresar en *bytes/seg*.

Ejemplo: la siguiente memoria permite realizar una operación de *lectura* y una operación de *escritura* simultáneas en la misma dirección o en direcciones diferentes.



Nota: $(\alpha) = 0 \rightarrow$ se realiza una operación de escritura

$(\beta) = 0 \rightarrow$ se realiza una operación de lectura

Aplicaciones de las memorias:

- ✓ Guardar datos durante periodos de tiempo "*pequeños*" (*volátiles*) o bien durante periodos de tiempo "*grandes*" (*no volátiles*)
- ✓ Implementar funciones lógicas
- ✓ Generar señales binarias complejas
- ✓ Hacer de *interfaz* entre dispositivos que envían y esperan recibir datos (bits) a diferente velocidad.
- ✓ etc.

Ejemplo de implementación de funciones lógicas utilizando una memoria semiconductora

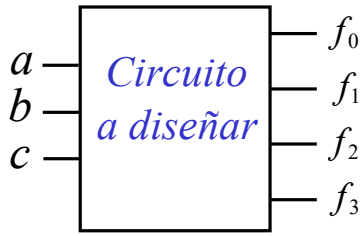
En este caso, se trata de implementar las siguientes funciones utilizando una memoria,

$$f_0(c,b,a) = \sum_3 (1,2,4)$$

$$f_1(c,b,a) = \sum_3 (0,4,5,6,7)$$

$$f_2(c,b,a) = \sum_3 (0,2,3,7)$$

$$f_3(c,b,a) = \sum_3 (1,4,6,7)$$



en la siguiente página se indica una posible solución

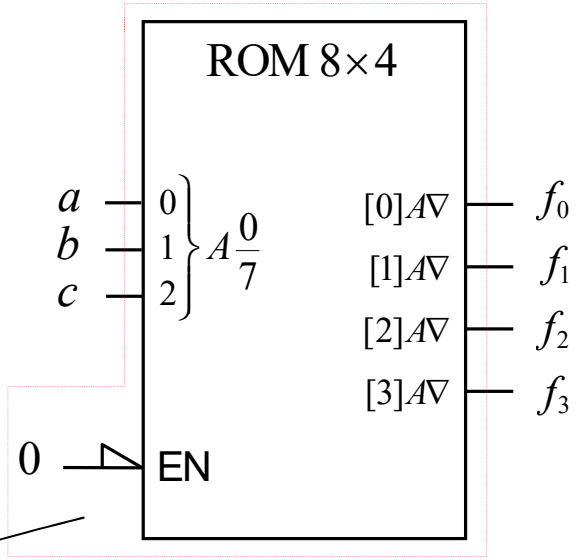
$$f_0(c,b,a) = \sum_3 (1,2,4)$$

$$f_1(c,b,a) = \sum_3 (0,4,5,6,7)$$

$$f_2(c,b,a) = \sum_3 (0,2,3,7)$$

$$f_3(c,b,a) = \sum_3 (1,4,6,7)$$

Nota: antes de poner en funcionamiento la memoria hay que guardar en ella la información adecuada. En los terminales de dirección se aplican las señales (c,b,a) presentes en las entradas del circuito a construir. La memoria proporciona en sus terminales de datos los valores que guarda en la dirección que indiquen en cada momento los valores de las entradas c, b y a (los valores proporcionados deben corresponder a los valores de las funciones f_3, f_2, f_1 y f_0 a implementar).



está memoria lee constantemente el valor guardado en la dirección indicada por el valor de las señales c, b y a

direcciones *contenido memoria*

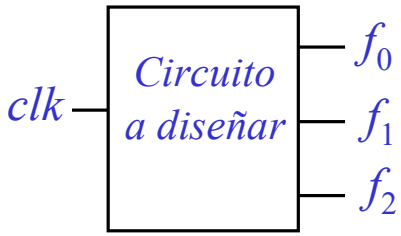
c	b	a	f_0	f_1	f_2	f_3
0	0	0	0	1	1	0
0	0	1	1	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	0
1	0	0	1	1	0	1
1	0	1	0	1	0	0
1	1	0	0	1	0	1
1	1	1	0	1	1	1

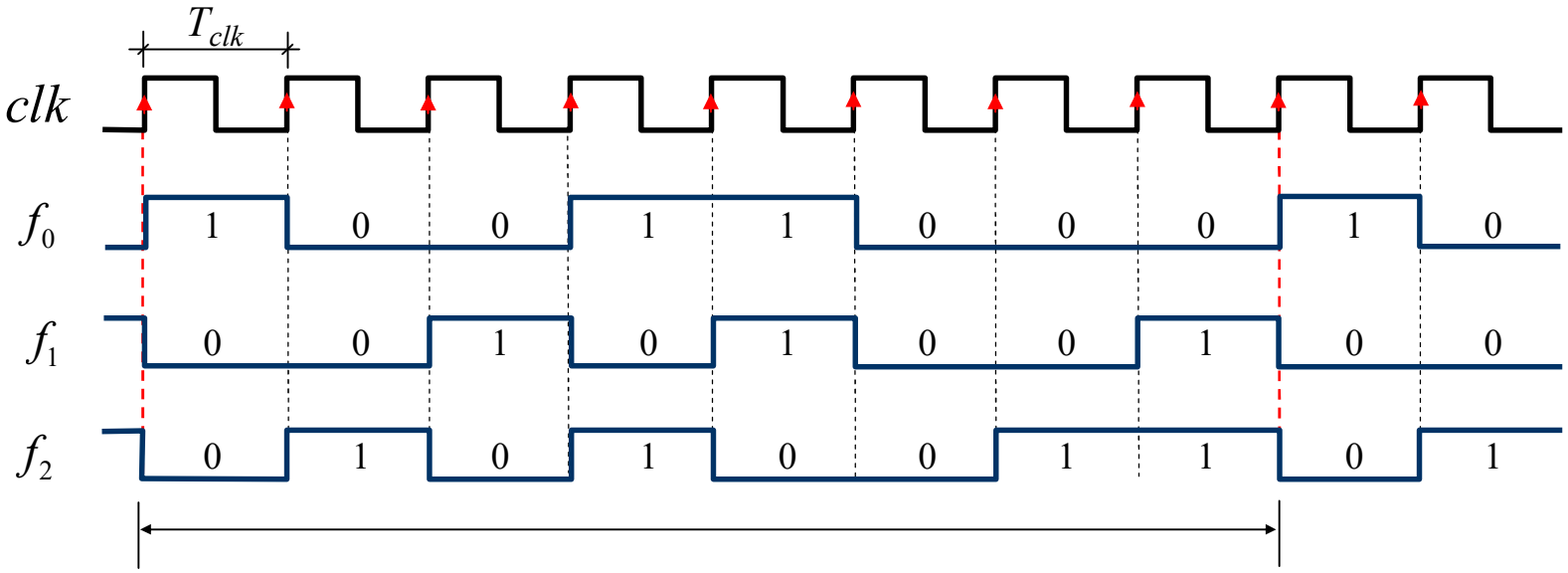
$f_2(0,1,0)$

contenido de la memoria (valores guardados en la memoria)

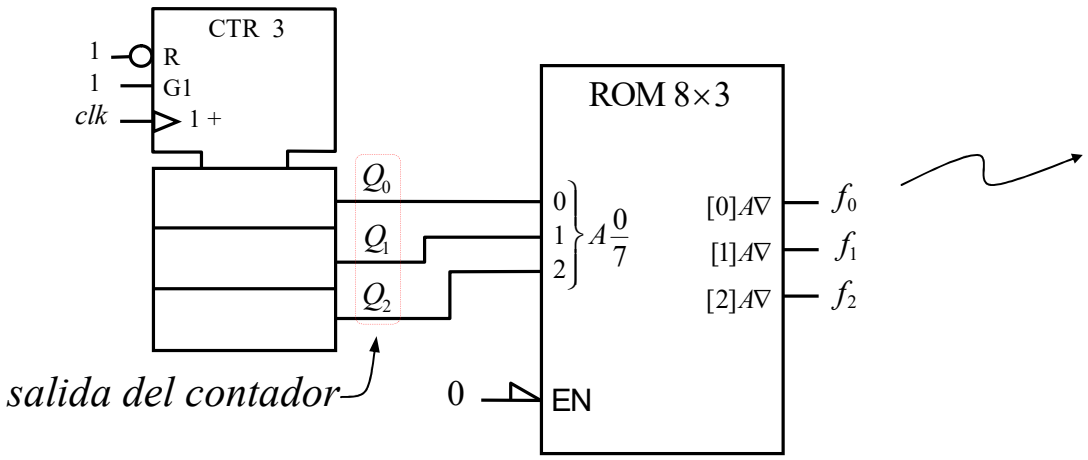
Ejemplo de generación de señales binarias periódicas utilizando una memoria

En este caso, se trata de diseñar un circuito que genere las siguientes señales *periódicas*,



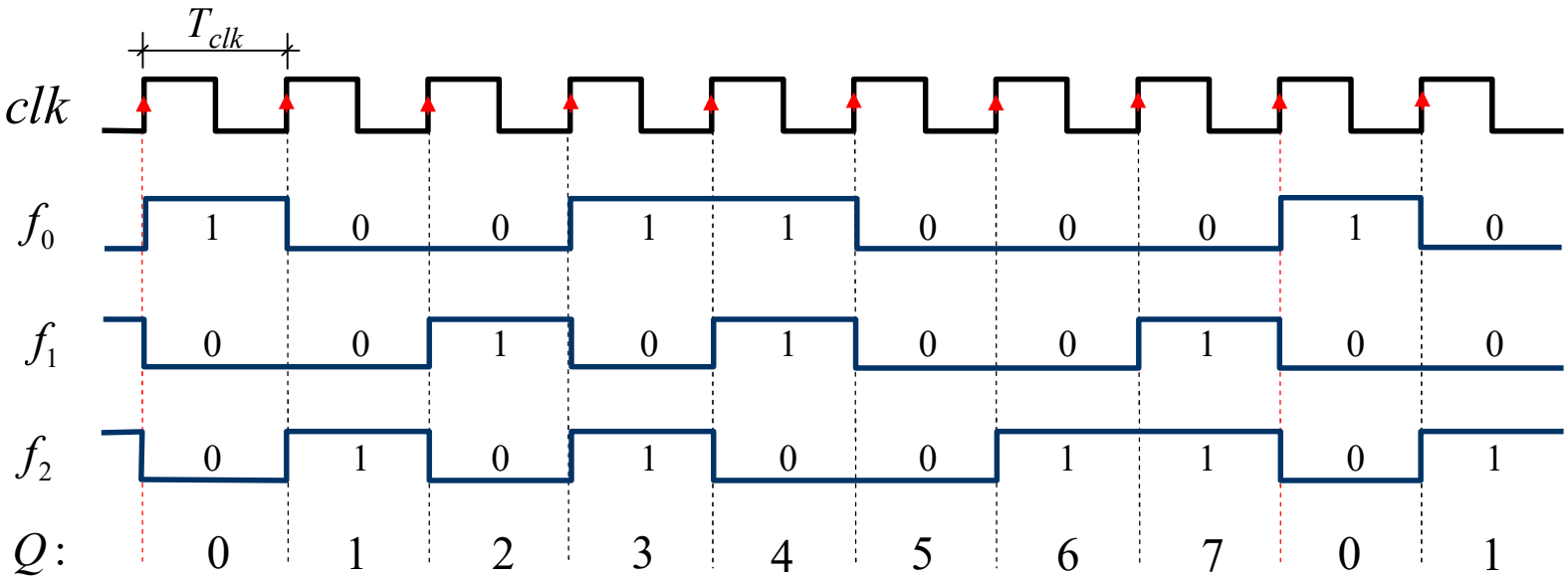


en la siguiente página se indica una posible solución



dirección	contenido
000	001
001	100
010	010
011	101
100	011
101	000
110	100
111	110

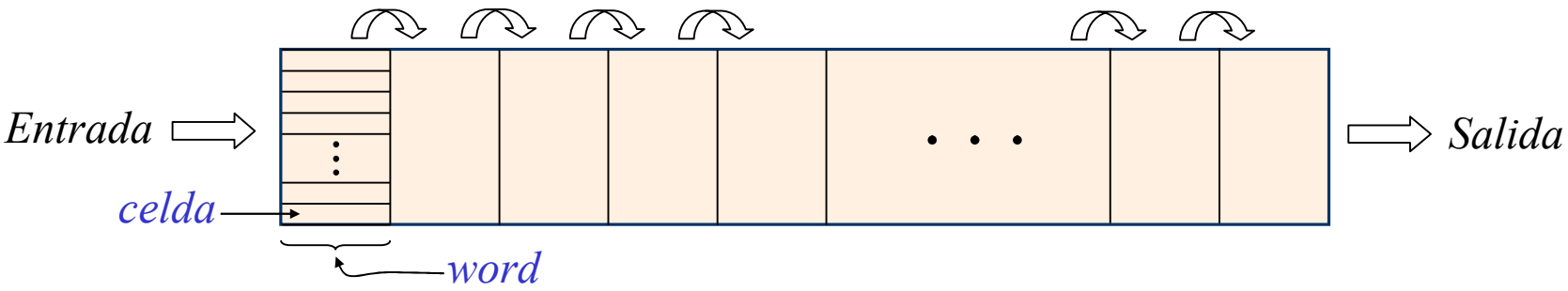
Nota: la memoria lee constantemente el valor guardado en la dirección indicada por la salida del contador



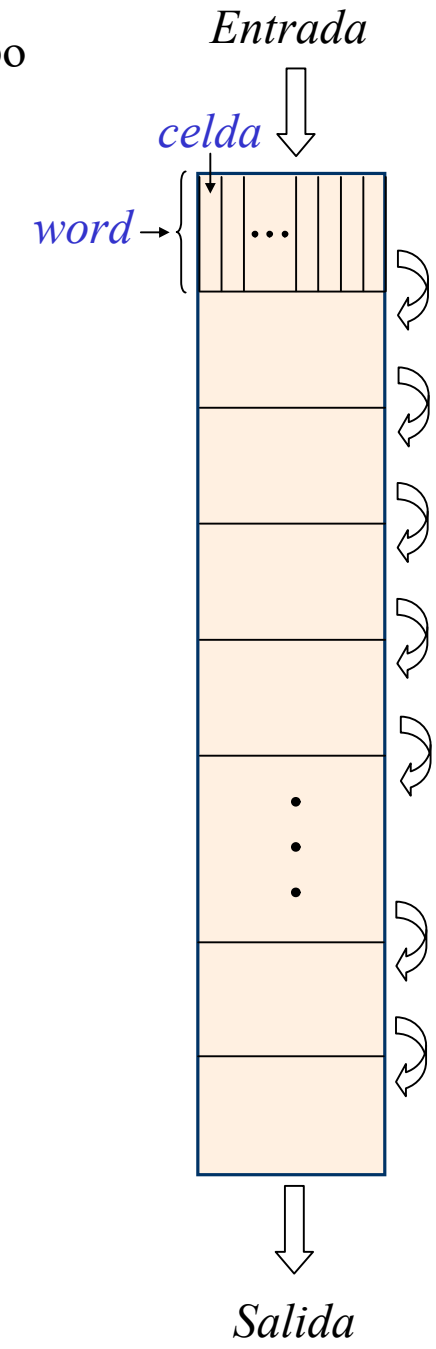
valor de la salida del contador en base 10

Memorias de acceso secuencial: *no se tiene un acceso directo a las posiciones de memoria*

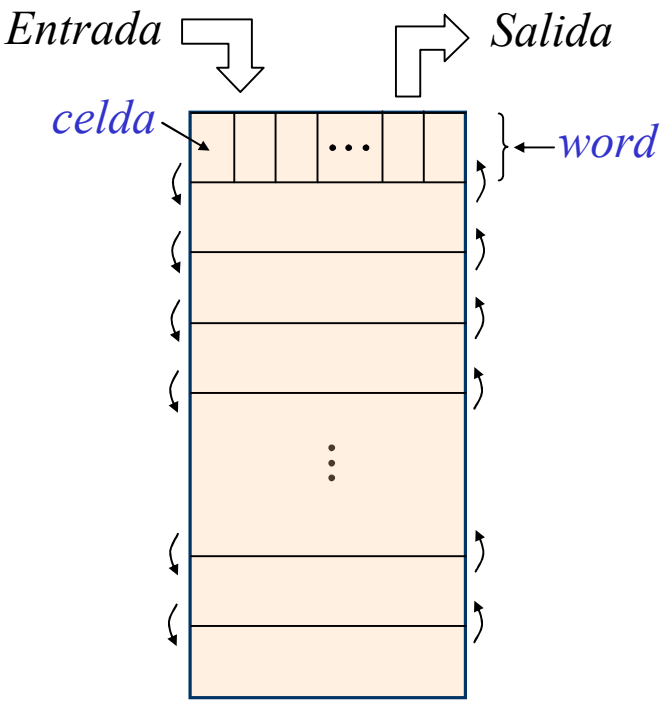
FIFO (*First In First Out*) o *tubo*



FIFO (*First In First Out*) o tubo



LIFO (*Last In First Out*) *o Pila*



FIN